CHEAPESTGO · V2 MIGRATION · ENGINEERING REFERENCE

V2 Gap — Implementation Plan

All features present in V1 (Next.js monolith) that are absent or stub-only in V2, across both the Express API and the Next.js App, with step-by-step implementation guidance.

90+ total gaps | API 56 | Frontend 40+ | 5 API Critical | 10 API Significant | 17 Admin endpoints | 7 Mobile endpoints | 8 FE feature groups | ~35 FE components

PART 1 OF 2

API — cheapestgo-api-v2

56 gaps · Express.js routes

TIER 1 Critical

Core booking pipeline breaks without these

01 **POST** `/api/webhooks/flight-supplier`

Async events from Duffel & Mystifly — ticket issued, refund resolved, booking updated

FILES

`src/routes/webhooks.route.ts`

STEPS

1. Add `router.post('/flight-supplier', express.raw({ type: '*/*' }), handler)` — raw body required for HMAC verification.
2. Verify Duffel signature: `X-Duffel-Signature: t=<ts>,v1=<hex>`. Reconstruct with `HMAC-SHA256(DUFFEL_WEBHOOK_SECRET, ts + '.' + rawBody)` using Node's built-in `crypto`.
3. Handle `order.updated`: re-fetch via `getDuffelOrder(orderId)`, update `flight_bookings.status`.
4. Handle `order_cancellation.confirmed`: mark `cancelled`; if Stripe refund not yet fired, call `stripe.refunds.create()`.
5. Handle Mystifly async callback (form-encoded): parse status, update `flight_bookings.status` to `ticketed` or `cancelled`.
6. Add `DUFFEL_WEBHOOK_SECRET` to `config.ts`. Return `200` immediately before async work to prevent Duffel retries.

DEPENDENCIES

`crypto` `stripe` `prisma` `getDuffelOrder()` `DUFFEL_WEBHOOK_SECRET`

●●●○○ Medium

02 **POST** `/api/internal/create-booking`

Atomically create flight_booking from booking_session after payment succeeds

FILES

`NEW · src/routes/internal.route.ts` `src/app.ts`

STEPS

1. Create `src/routes/internal.route.ts`. Auth: `Authorization: Bearer <INTERNAL_SECRET>`. Mount in `app.ts`: `app.use('/api/internal', internalRouter)`.

2. Accept `{ sessionId }`. Query `booking_sessions`; throw `404` if missing or already `completed_at IS NOT NULL`.
3. Use `prisma.$transaction`: SELECT FOR UPDATE the session row, verify `completed_at IS NULL`, then insert into `flight_bookings`. Prevents double-booking on concurrent Stripe webhook retries.
4. Mystify bookings: call `mystiflyRequest('AirBook', ...)`. Duffel: order was created at `/flights/book` — just persist the record from session data.
5. Mark `completed_at = NOW()` and return the created `flight_booking.id`.

DEPENDENCIES

`prisma.$transaction` `mystiflyRequest()` `INTERNAL_SECRET`

●●●●○ High

---

03 **POST** `/api/internal/issue-ticket`

Extract Duffel e-ticket from order, update flight_bookings to ticketed

FILES

`src/routes/internal.route.ts`

STEPS

1. Accept `{ bookingId }`. If `status === 'ticketed'` return early (idempotent).
2. Call `GET https://api.duffel.com/air/orders/{provider_order_id}` via `duffelHeaders()`. Check `data.documents` — non-empty means tickets are issued.
3. Extract `data.documents[].document_number`. Store as JSON in `ticket_numbers` column (add migration if missing).
4. Update: `SET status = 'ticketed', ticketed_at = NOW(), ticket_numbers = $1`. Optionally send confirmation email via Resend.

DEPENDENCIES

`duffelHeaders()` `prisma` `RESEND_API_KEY`

●●○○○ Low

---

04 **POST** `/api/internal/auto-recover`

Detect payment-succeeded / booking-missing mismatches and re-trigger booking creation

FILES

`src/routes/internal.route.ts`

STEPS

1. Query `booking_sessions WHERE payment_intent_id IS NOT NULL AND completed_at IS NULL AND created_at < NOW() - INTERVAL '10 minutes'`.
2. For each candidate: `stripe.paymentIntents.retrieve(pi_id)`. Skip if status ≠ `succeeded`.
3. Check if `flight_bookings` row already exists for the session. If yes, mark session complete and skip.
4. If payment succeeded but no booking: run the create-booking logic inline. Create a `notifications` row for ops audit.

DEPENDENCIES

`stripe` `prisma` `INTERNAL_SECRET`

●●●○○ Medium

---

05 **POST** `/api/admin/bookings/:id`

Admin booking actions — force-cancel, refund, restore, retry, recheck, resend email

FILES

`src/routes/admin.route.ts`

STEPS

1. `force_cancel` : call `flightsService.cancelBooking()` bypassing user ownership check.
2. `refund` : accept `{ amount, reason }` . `stripe.refunds.create({ payment_intent, amount: amount * 100 })` . Update status to `refunded` .
3. `restore` : set `status = 'confirmed'` for incorrectly cancelled bookings.
4. `recheck` : call `getDuffelOrder()` or TGX booking detail, sync status back to DB.
5. `resend_email` : accept `{ type }` . Re-call Resend with the booking confirmation or cancellation template.
6. `cancel_awaiting_ticket` : Duffel cancel flow + full Stripe refund for stuck `awaiting_ticket` bookings.

DEPENDENCIES

`flightsService` `stripe` `getDuffelOrder()` `RESEND_API_KEY`

●●●●○ High

TIER 2 Significant

User-visible API features

#	ENDPOINT	FILE(S)	IMPLEMENTATION NOTE
06	GET /api/invoice/:id/pdf	NEW · src/routes/invoice.route.ts	Install pdfkit. Auth required; verify booking belongs to user. Query flight/hotel booking, build PDF with PDFKit (doc.pipe(res)), set Content-Type: application/pdf and Content-Disposition: attachment.
07	GET /api/weather	NEW · src/routes/weather.route.ts	Use free **open-meteo.com** (no API key). Call /v1/forecast?latitude&longitude&t=temperature_2m,weathercode&daily=.... In-memory cache per coordinate (30 min TTL). Normalize to { current, daily[] }.
08	POST /api/flights/price-calendar-live	flights.route.ts	Accept { origin, destination, dates[], cabinClass, adults } (max 7 dates). Run Promise.allSettled(dates.map(d => searchFlights({...}))). Return cheapest offer per date.
09	POST /api/stays/quote & /book	NEW · src/routes/stays.route.ts NEW · src/lib/stays/duffel-stays.ts	Wrap Duffel Stays API: /stays/search_results → /stays/quotes → /stays/bookings. Quote: accept propertyId + dates, return quoteId + price. Book (auth required): verify Stripe PI, call Duffel, persist bookings row with provider='duffel_stays'.
10	GET /api/photos/hotel & /destination	photos.route.ts	Hotel photo: DB → TGX GraphQL → Google Places fallback chain. Destination photo: IATA → city-name map → Google Places. Both: Cache-Control: public, max-age=86400; return inline placeholder SVG if all fallbacks fail.
11	POST /api/hotels/autocomplete/resolve	hotels.route.ts	Accept { city }. Check tgx_destinations cache table first. On miss, call TGX destination lookup mutation. Cache result. Return { code, type, name }.
12	POST /api/hotels/search/stream	hotels.route.ts	SSE: set Content-Type: text/event-stream, flush headers. Run TGX search; emit { type:'hotels', hotels:[] } per page. Run Duffel Stays in parallel; emit same shape. Emit { type:'done' }. Handle req.on('close') to abort via AbortController.

#	ENDPOINT	FILE(S)	IMPLEMENTATION NOTE
13	Mystifly: `ticket-display`, `trip-details`, `booking-notes`	`flights.route.ts`	ticket-display + trip-details: replace 503 stubs with `mystiflyRequest('AirTicketDisplay'|'AirTripDetails', ...)` when live key available. Booking notes: DB CRUD on `flight_booking_notes` table — no Mystifly call needed, implement now.
14	`POST /api/internal/retry-emails`	`internal.route.ts`	Query `email_logs WHERE status='failed' LIMIT 50`. Re-POST each to Resend API with stored payload. Update status on success.
15	`POST /api/internal/refresh-flights`	`internal.route.ts`	Accept `{ origin, destination, departureDate, cabinClass }`. Call `searchFlights()` — cache warm is the side effect. Return result count.

TIER 3 Admin Endpoints

17 missing operator endpoints — all in admin.route.ts

#	ROUTE	IMPLEMENTATION
16–17	`GET/POST /admin/notifications`	GET: query `notifications WHERE user_id IS NULL ORDER BY created_at DESC` paginated, return unread count. POST actions: `mark_read` (by id), `mark_all_read` (all unread).
18–19	`GET/POST /admin/destinations`	GET: paginated `popular_destinations` table. POST actions: `create`, `update` (patch by id), `delete`, `reorder` (batch sort_order). Create migration if table missing: id, name, iata, image_url, active, sort_order.
20–21	`GET/POST /admin/price-alerts`	GET: all `price_alerts`, filterable by `status`, `origin`, `destination`, paginated 25/page. POST actions: `activate`, `deactivate` (toggle is_active), `delete`.
22–23	`GET/POST /admin/reviews`	GET: `hotel_review_items` with hotel name join via `hotel_content`, paginated. POST action `delete`: hard delete by id, then recalculate aggregate in `hotel_reviews`.
24–25	`GET/POST /admin/saved-trips`	GET: all `saved_trips` with user email join, grouped by type for stats, paginated. POST action `delete`: hard delete by id.
26–27	`GET/POST /admin/communication`	GET: `email_logs ORDER BY created_at DESC`, filter by booking_id, status, type, paginated 50/page. Create migration if missing: id, booking_id, to, type, status, resend_id, error, created_at. POST action `retry`: re-send via Resend, update status.
28	`GET /admin/search`	Global search via `?q=` across bookings (booking_id/email), users (email), flight_bookings (PNR). Return grouped results with type labels.
29	`POST /admin/settings`	Upsert key/value pairs into `admin_settings` table. Keys: `otv_credit_limit`, `duffel_balance_alert_threshold`, `maintenance_mode`, mobile version fields.
30	`GET /admin/stripe`	`stripe.balance.retrieve()` + `paymentIntents.list(limit:20)` + `refunds.list(limit:20)` + `disputes.list(limit:10)`. Aggregate into single dashboard payload.
31–32	`GET/POST /admin/mobile`	GET: flight booking count last 30d, device_registrations count, API key status (redacted), version config from admin_settings. POST actions: `rotate_api_key` (generate UUID, store hashed), `update_version` (upsert min/latest/force_update).

TIER 4 Mobile App Layer

7 endpoints — Expo app surface entirely absent from V2

33–39	GET	POST	`/api/mobile/*`

Decide auth strategy first — JWT reuse (recommended) or X-Mobile-Api-Key

FILES

`NEW · src/routes/mobile.route.ts` `src/middleware/auth.middleware.ts`

AUTH DECISION

A **JWT (recommended)**: update Expo app to send `Authorization: Bearer <jwt>`. Mobile endpoints reuse `requireAuth`. Simpler long-term.

B **Mobile API Key**: add `requireMobileApiKey` middleware checking `X-Mobile-Api-Key` against hashed value in `admin_settings`. No Expo code changes needed.

ENDPOINTS

ROUTE	IMPLEMENTATION
`GET /mobile/version-check`	Read `min_version`, `latest_version`, `force_update` from admin_settings. Public — no auth.
`GET /mobile/landing`	`flight_deals` (limit 6) + `hotel_deals` (limit 6) + `popular_destinations` (limit 8). Public.
`GET /mobile/trips`	Auth required. `bookings + flight_bookings` for current user, sorted by created_at DESC.
`POST /mobile/register-device`	Auth required. Upsert into `device_registrations(user_id, expo_push_token, platform)`. Create table via migration.
`POST /mobile/log`	Accept log entry array. Write to stdout as structured JSON. Rate-limit by IP. No auth.
`POST /mobile/flights/book`	Same logic as `/api/flights/book`. Returns session ID + Stripe PI client secret.
`POST /mobile/flights/confirm`	Same logic as `/api/flights/confirm`. Auth required.

●●●○○ Medium (auth design first · endpoints straightforward)

PART 2 OF 2

Frontend — cheapestgo-app-v2

40+ gaps · Next.js 15 App Router

FE TIER 1 **Checkout & Cancellation** — Missing components break core conversion flows

FE-01 COMP `src/app/checkout/ — 9 missing components`

V1 checkout has 12 components; V2 has 3. Vouchers, success screen, Stripe embedded, special requests all absent.

MISSING COMPONENTS

`VoucherInput.tsx` `AvailablePromos.tsx` `StripeEmbeddedCheckout.tsx` `BookingSuccess.tsx` `CancellationPolicySection.tsx` `SpecialRequestsSection.tsx` `BookingForSection.tsx` `SubmitBookingButton.tsx`

STEPS

1 **VoucherInput**: text field + "Apply" button. POST to `/api/vouchers/validate` with code + totalAmount. On success, display discount amount and update price total in parent state.

2 **AvailablePromos**: fetch `/api/vouchers/list` on mount. Display active vouchers as one-click chips. Clicking a chip fires the same validate flow as VoucherInput.

3 **StripeEmbeddedCheckout**: use `@stripe/react-stripe-js` `<EmbeddedCheckout>` component. Receive `clientSecret` from parent (from hotel `create-payment` API call). Handle `onComplete` to trigger booking confirm.

4. **BookingSuccess**: rendered after `/hotels/confirm` returns OK. Show booking ID, property name, dates, download invoice button (links to `/api/invoice/:id/pdf?type=hotel`).
5. **CancellationPolicySection**: display `refundableTag` and `free_cancel_deadline` from the prebook response. Show as a green "Free cancellation until..." or amber "Non-refundable" badge.
6. **SpecialRequestsSection**: textarea bound to form state. Passed to `/hotels/confirm` as `specialRequests`.

API CALLS

`POST /api/vouchers/validate` `GET /api/vouchers/list` `POST /api/hotels/confirm` `GET /api/invoice/:id/pdf`

High

FE-02 COMP `src/app/trips/ — Cancellation flow + Trip Map`

V1 trips has 8 components; V2 has 3. Self-service cancel, fee display, modification modal, trip map all absent.

MISSING COMPONENTS

`CancellationModal.tsx` `CancellationFeeCard.tsx` `CancellationPolicies.tsx` `ModificationModal.tsx` `TripMapView.tsx` `FlightBookingCard.tsx`

STEPS

1. **CancellationModal**: two-step flow. Step 1 — fetch `POST /api/flights/cancel-quote` or show hotel policy; display refund amount and penalty. Step 2 — user confirms; call `POST /api/flights/cancel-booking` or `POST /api/hotels/cancel`.
2. **CancellationFeeCard**: display refundAmount, penaltyAmount, currency. Show as a before/after price breakdown. Used inside CancellationModal step 1.
3. **TripMapView**: dynamic import `SearchMapContainer` with the destination coordinates. Show a single destination pin + nearby POIs. Renders inside `/trips/[id]` below the booking detail.
4. **FlightBookingCard**: dedicated card variant for flight bookings showing airline, route, segments, cabin class, PNR, status badge. Differentiates from hotel booking card in the trips list.

API CALLS

`POST /api/flights/cancel-quote` `POST /api/flights/cancel-booking` `POST /api/hotels/cancel`

Medium

FE TIER 2 Property Page

V1 has ~22 property sub-components; V2 has 5

FE-03 COMP `src/app/property/[id]/ — 14 missing components`

Booking widget, date picker, mobile CTA, policies, FAQ, weather, reviews modal, room card all absent

MISSING (IMPLEMENT IN PRIORITY ORDER)

`BookingWidget.tsx` `PropertyDatePicker.tsx` `MobileBookingCTA.tsx` `MobilePropertyHeader.tsx` `RoomCard.tsx` `RoomsAvailabilitySection.tsx` `PropertyNav.tsx` `PoliciesSection.tsx` `FAQSection.tsx` `WeatherWidget.tsx` `ReviewsModal.tsx` `ReviewsSummary.tsx` `LocationSection.tsx` `PoiDetailsModal.tsx`

STEPS

1. **BookingWidget** (desktop sidebar): sticky `position: sticky; top: 80px` container. Shows price per night, date range picker trigger (delegates to PropertyDatePicker), room count selector, "Reserve" CTA that routes to `/checkout?...params`.
2. **PropertyDatePicker**: inline calendar using the existing `DatePicker` component from the landing hero. Reads/writes from `useSearchStore` dates state. Shows available date ranges.
3. **MobileBookingCTA**: fixed bottom bar on mobile (`position:fixed; bottom:0; left:0; right:0`). Shows price/night + "Reserve" button. Hidden on desktop via `lg:hidden`.

4. **RoomsAvailabilitySection**: fetch `POST /api/hotels/prebook` with the propertyId + dates. Show RoomCard for each available rate. Handle loading skeleton during prebook fetch.
5. **PropertyNav**: sticky anchor nav bar (Overview, Rooms, Reviews, Location). Uses `IntersectionObserver` to highlight active section as user scrolls.
6. **WeatherWidget**: fetch `GET /api/weather?lat=&lng=` using the property's coordinates. Show current temp + 5-day forecast as small icons. Cache in component state.
7. **PoliciesSection + FAQSection**: render property `policies` array and static FAQ from the property data. Accordion pattern using `details/summary` or a simple toggle state.

API CALLS

`POST /api/hotels/prebook` `GET /api/weather` `GET /api/photos/hotel`

●●●●○ High

FE TIER 2 Search & Map

V1 search has 17 components; V2 has 3

FE-04 COMP `src/app/search/ — Map view + Filters`

Map/list split view, search filters, mobile search modal all absent

MISSING

`MapResultsClient.tsx` `SearchMapView.tsx` `SearchFilters.tsx` `ActiveFiltersSummary.tsx` `CountryCityPicker.tsx` `MobileSearchModal.tsx` `SearchLoadingSkeleton.tsx` `ResponsiveSearchHeader.tsx`

STEPS

1. **SearchMapView**: `dynamic()` import of `SearchMapContainer` with `ssr:false`. Left panel: scrollable hotel card list. Right panel: sticky map. Toggle between list and map on mobile. Already fully implemented in V1 — port `SearchMapView.tsx` directly.
2. **SearchFilters**: price range slider, star rating checkboxes, board type chips, refundable toggle, property type chips. State managed via `useSearchFilters()` from search store (already exists in V2 store).
3. **ActiveFiltersSummary**: renders one removable chip per active filter above the results list. Clicking a chip removes that filter from store state.
4. **SearchLoadingSkeleton**: array of 6 skeleton cards with animated pulse. Show while the SSE stream is open (before first hotels arrive).
5. **Map toggle**: add `?view=map|list` URL param handling to the search page. Conditionally render `SearchMapView` or `HotelResultsClient`.

●●●●○ High

FE-05 COMP `src/app/flights/ — Seat map, bags, price calendar, price alert`

Advanced flight booking components absent from V2

MISSING

`SeatMapPanel.tsx` `BagSelectionPanel.tsx` `PriceCalendar.tsx` `PriceAlertButton.tsx` `DuffelFareConditions.tsx`

STEPS

1. **BagSelectionPanel**: fetch `POST /api/flights/bags` with offerId on booking page mount. Render bag options as quantity selectors. Sum selected bags' prices into the total.
2. **SeatMapPanel**: fetch `POST /api/flights/seat-map`. Render a grid: available seats (clickable), occupied (greyed), exit rows (labelled). Seat selection state persists into booking payload.
3. **PriceCalendar**: fetch `GET /api/flights/price-calendar?origin&destination&month&cabin`. Render a calendar grid with price labels per day. Color-code cheap (green) vs expensive (red). Clicking a date updates the search params.
4. **PriceAlertButton**: toggle button. On enable: POST to `/api/bookings/price-alerts` with current route + cabin. Show toast confirmation. On page load, check if alert already active for this route.

API CALLS

`POST /api/flights/bags` `POST /api/flights/seat-map` `GET /api/flights/price-calendar` `POST /api/bookings/price-alerts`

●●●○○ Medium

FE TIER 3 Auth Modal, Admin & UI Primitives

Enhancement tier — lower user impact

COMP Auth Modal Flow — in-page auth without navigation

V1: 13 components · V2: 4

MISSING

`AuthModal.tsx` `AuthModalWrapper.tsx` `EmailStep.tsx` `PasswordStep.tsx` `RegisterStep.tsx` `ForgotPasswordStep.tsx` `VerifyEmailStep.tsx` `PasswordRequirements.tsx`

V2 currently hard-navigates to `/login` and `/register`. To restore the in-page modal: create a modal overlay component that wraps the multi-step flow. Connect to a global `useAuthModal()` Zustand slice. Trigger from any page's "Sign in" link instead of navigating away.

COMP Admin Dashboard Charts & Command Palette

V1: 14 components · V2: 3

MISSING

`CommandPalette.tsx` `RevenueChart.tsx` `DuffelInsightsCharts.tsx` `ConversionFunnel.tsx` `ProviderIntegrations.tsx` `TopRoutes.tsx` `TopNav.tsx` `MobileAdminNav.tsx`

Charts: use `recharts` (already a common dep). RevenueChart — line chart from `GET /admin/stats`. DuffelInsightsCharts — bar chart from Duffel balance history. CommandPalette — keyboard shortcut (`⌘K`) overlay for navigating admin sections, built on a simple filtered list + keyboard event listener.

COMP Missing UI Primitives

V1: 24 · V2: 14 — 10 missing

MISSING FROM SRC/COMPONENTS/UI/

`alert-dialog.tsx` `avatar.tsx` `dropdown-menu.tsx` `sheet.tsx` `table.tsx` `logo.tsx` `optimized-image.tsx` `animations.tsx`

All are standard shadcn/ui components. Run `pnpm dlx shadcn@latest add alert-dialog avatar dropdown-menu sheet table` to scaffold them. `Logo.tsx`, `OptimizedImage.tsx`, and `Animations.tsx` are custom — port directly from V1 source.

COMP Common Utilities

5 components absent from V2

MISSING FROM SRC/COMPONENTS/COMMON/

`BackButton.tsx` `ClientOnly.tsx` `CurrencySelector.tsx` `FormDatePicker.tsx` `SaveButton.tsx`

BackButton: calls `router.back()` or falls back to a provided `href`. Used widely across property, trips, and checkout pages. **CurrencySelector**: dropdown of currencies that writes to `useUserCurrency()` store and persists to user preferences. **ClientOnly**: renders children only after hydration — wraps map components to suppress SSR mismatch. All are small, port directly from V1.

COMP Landing Sections & AI Search

5 sections + entire AI search overlay missing

MISSING LANDING SECTIONS

`CuratedSection.tsx` `ExploreUniqueStays.tsx` `GuestFavoritesSection.tsx` `HotelDealsSection.tsx` `LastMinuteWeekendDeals.tsx`

MISSING AI SEARCH

`AISearchBar.tsx` `AIPromptInput.tsx` `AIResultsPreview.tsx` `SearchModeToggle.tsx` `AISuggestionChips.tsx`

Landing sections: port directly from V1 — they are display-only components with no functional dependency gaps. AI search requires the `POST /api/voice` backend endpoint to be implemented first; defer until voice is in scope.

EXCLUDED Out of Scope

ITEM	REASON
`POST /api/voice`	AI assistant — not in V2 scope. Blocked on AI search UI too.
`Mystifly void/reissue/refund`	V2 has 503 stubs. Implement when live Mystifly key is issued.
`/auth/auth-code-error page`	V2 should redirect to `/login?error=oauth_failed` and handle it there.
`/dev-policy-test, /meta-preview, /test-map`	Dev-only — add locally ad hoc.
`VoiceAssistant component`	Depends on voice API endpoint. Defer.
`MapboxDirections / TripRouteLayer`	Needed only for trip itinerary feature — not on the critical path.
`MagneticButton / TiltCard`	Animation flourishes — not functional requirements.